

目录

后缀自动机	1
状态究竟表示什么	3
常用结论	4
与另一个串的最长公共子串	4
字典序第 k 小不同子串	5
后缀链接树与出现位置	6
AC 自动机	7
回文自动机 (PAM)	8
字符串最小表示法	10
Manacher	11
Z 函数	11
常见推论	12
KMP	12
border、周期与失配树	13
后缀数组与 LCP	14
Lyndon 分解	15
通配符	16
广义后缀自动机	17
双模字符串哈希	20
异或哈希与可交换哈希	21
最重要的边界: XOR 只保留奇偶	22
前缀 XOR: 区间奇偶集合	22
图的割边集合签名	23
无序树和多重子结构不能裸 XOR	23
什么时候不要用	23

后缀自动机

固定小写字母表。状态 0 是初始状态; 每次 `extend` 后的 `last` 表示整个当前字符串。不同子串数量为所有非初始状态的 `length-link.length` 之和。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class SuffixAutomaton {
public:
    struct State {
        array<int, 26> next{};
        int link = -1;
        int length = 0;
        long long occurrences = 0;
    };
};
```

```

};

private:
    vector<State> states;
    int last = 0;

public:
    SuffixAutomaton() {
        states.push_back(State{});
    }

    void extend(char character) {
        int letter = character - 'a';
        assert(0 <= letter && letter < 26);
        int current = (int)states.size();
        states.push_back(State{});
        states[current].length = states[last].length + 1;
        states[current].occurrences = 1;

        int prefix = last;
        while (prefix != -1 && states[prefix].next[letter] == 0) {
            states[prefix].next[letter] = current;
            prefix = states[prefix].link;
        }
        if (prefix == -1) {
            states[current].link = 0;
        } else {
            int target = states[prefix].next[letter];
            if (states[prefix].length + 1 == states[target].length) {
                states[current].link = target;
            } else {
                int clone = (int)states.size();
                states.push_back(states[target]);
                states[clone].length = states[prefix].length + 1;
                states[clone].occurrences = 0;
                while (prefix != -1 &&
                    states[prefix].next[letter] == target) {
                    states[prefix].next[letter] = clone;
                    prefix = states[prefix].link;
                }
                states[target].link = clone;
                states[current].link = clone;
            }
        }
        last = current;
    }

    long long distinct_substring_count() const {
        long long answer = 0;
        for (int state = 1; state < (int)states.size(); ++state) {
            answer += states[state].length -
                states[states[state].link].length;
        }
    }

```

```

        return answer;
    }

    vector<long long> occurrence_counts() const {
        vector<int> order(states.size());
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int lhs, int rhs) {
            return states[lhs].length > states[rhs].length;
        });
        vector<long long> count(states.size());
        for (int state = 0; state < (int)states.size(); ++state) {
            count[state] = states[state].occurrences;
        }
        for (int state : order) {
            if (states[state].link != -1) {
                count[states[state].link] += count[state];
            }
        }
        return count;
    }

    int transition(int state, char character) const {
        return states[state].next[character - 'a'];
    }

    int last_state() const {
        return last;
    }

    const vector<State>& data() const {
        return states;
    }
};

```

构造复杂度为 $O(n)$ (上面的计数排序为简洁使用了比较排序, 统计出现次数时为 $O(n \log n)$)。克隆状态的初始出现次数必须为 0。多个字符串的广义 SAM 不能简单地在每个字符串前把 `last` 设回根后直接套此模板; 应先建 Trie, 再按 BFS 扩展。

状态究竟表示什么

对于一个子串 t , 记 $\text{endpos}(t)$ 为它在原串中所有出现位置的右端点集合。SAM 把 endpos 完全相同的子串放入同一个状态。对非根状态 v :

- $\text{length}[v]$ 是该状态中最长子串的长度;
- 最短长度是 $\text{length}[\text{link}[v]] + 1$;
- 它恰好代表长度区间

$$[\text{length}(\text{link}(v)) + 1, \text{length}(v)]$$

中的一段连续后缀, 这些串的出现位置集合相同;

- 汇总后的 $\text{occurrences}[v]$ 就是 endpos 大小, 因此状态内所有子串出现次数相同;
- $\text{endpos}(v) \setminus \text{endpos}(\text{link}[v])$ 。把后缀链接反向建边得到后缀链接树, 祖先对应更短且出现位置更多的后缀类。

这解释了为什么每个状态贡献

```
states[v].length - states[states[v].link].length
```

个不同子串，而不是只贡献一个。

常用结论

设 `count = automaton.occurrence_counts()`:

问题	SAM 上的做法
模式是否为子串	从根沿字符转移；缺边即不存在
模式出现次数	走到状态 <code>v</code> 后答案为 <code>count[v]</code>
不同子串数	对每个非根状态累加 <code>length[v] - length[link[v]]</code>
新增一个字符产生多少新子串	本次新状态 <code>current</code> 贡献 <code>length[current] - length[link[current]]</code>
至少出现 k 次的最长子串	在 <code>count[v] >= k</code> 的状态中最大化 <code>length[v]</code>
所有不同子串的长度和	对每个状态把最短到最长长度作等差数列求和
字典序第 k 小不同子串	在转移 DAG 上统计每条出边后的路径数，再按字符贪心

所有不同子串长度之和可能达到 $\Theta(n^3)$ ，下面用 `__int128`:

```
__int128 distinct_substring_length_sum(const SuffixAutomaton& automaton) {
    const auto& states = automaton.data();
    __int128 answer = 0;
    for (int state = 1; state < (int)states.size(); ++state) {
        long long low = states[states[state].link].length + 1;
        long long high = states[state].length;
        answer += (__int128)(low + high) * (high - low + 1) / 2;
    }
    return answer;
}

int longest_substring_appearing_at_least(
    const SuffixAutomaton& automaton,
    const vector<long long>& count, long long minimum_occurrences) {
    const auto& states = automaton.data();
    assert(count.size() == states.size());
    int answer = 0;
    for (int state = 1; state < (int)states.size(); ++state) {
        if (count[state] >= minimum_occurrences) {
            answer = max(answer, states[state].length);
        }
    }
    return answer;
}
```

与另一个串的最长公共子串

在 SAM 上扫描另一个串，维护当前状态和当前连续匹配长度。缺少转移时沿后缀链接缩短；这不是把状态直接重置为根，因为较短后缀仍可能继续匹配。

```

string longest_common_substring(
    const SuffixAutomaton& automaton, string_view other) {
    const auto& states = automaton.data();
    int state = 0;
    int matched = 0;
    int best_length = 0;
    int best_end = -1;

    for (int index = 0; index < (int)other.size(); ++index) {
        int letter = other[index] - 'a';
        assert(0 <= letter && letter < 26);
        while (state != 0 && states[state].next[letter] == 0) {
            state = states[state].link;
            matched = states[state].length;
        }
        if (states[state].next[letter] != 0) {
            state = states[state].next[letter];
            ++matched;
        } else {
            matched = 0;
        }
        if (matched > best_length) {
            best_length = matched;
            best_end = index;
        }
    }
    return best_end == -1 ? string() :
        string(other.substr(best_end - best_length + 1, best_length));
}

```

多个串的最长公共子串：用第一个串建 SAM。对每个其余串按上面方法扫描，记录它在每个状态能匹配到的最大长度；再按 `length` 降序令

```
best[link[v]] = max(best[link[v]], min(best[v], length[link[v]]));
```

最后对每个状态取所有字符串 `best` 的最小值，再在所有状态中取最大值。传播时必须截断到父状态的 `length`。

字典序第 k 小不同子串

从根出发的每条非空路径唯一对应一个不同子串。令 `paths[v]` 为从状态 v 出发能形成的非空字符串数，则

$$\text{paths}(v) = \sum_{\substack{c \\ v \rightarrow u}} (1 + \text{paths}(u)).$$

转移一定走向 `length` 更大的状态，因此按 `length` 降序 DP。下面 k 从 1 开始，并把计数饱和在 `ULLONG_MAX`，避免只为比较大小而溢出。

```

optional<string> kth_distinct_substring(
    const SuffixAutomaton& automaton, unsigned long long k) {
    using u64 = unsigned long long;
    if (k == 0) return nullopt;
    const auto& states = automaton.data();

```

```

vector<int> order(states.size());
iota(order.begin(), order.end(), 0);
sort(order.begin(), order.end(), [&](int lhs, int rhs) {
    return states[lhs].length > states[rhs].length;
});

auto capped_add = [](u64 lhs, u64 rhs) {
    u64 maximum = numeric_limits<u64>::max();
    return maximum - lhs < rhs ? maximum : lhs + rhs;
};
vector<u64> paths(states.size());
for (int state : order) {
    for (int next : states[state].next) {
        if (next == 0) continue;
        paths[state] = capped_add(
            paths[state], capped_add(1, paths[next]));
    }
}
if (k > paths[0]) return nullopt;

string answer;
int state = 0;
while (true) {
    bool moved = false;
    for (int letter = 0; letter < 26; ++letter) {
        int next = states[state].next[letter];
        if (next == 0) continue;
        u64 block = capped_add(1, paths[next]);
        if (k > block) {
            k -= block;
            continue;
        }
        answer.push_back(char('a' + letter));
        if (k == 1) return answer;
        --k; // 跳过恰好等于当前 answer 的子串
        state = next;
        moved = true;
        break;
    }
    assert(moved);
}
}

```

若按出现次数重复计算子串，则每条转移 $v \rightarrow u$ 的当前字符串权重应从 1 改成 $\text{count}[u]$ ，并相应修改第 k 小过程；不要把“不同子串”和“按出现次数计数”混用。

后缀链接树与出现位置

建串时每次 `extend` 后记录 `last_state()`，第 i 个前缀状态代表结束位置 i 。把这些位置挂到对应状态，再在后缀链接树上做子树汇总，就得到每个状态的全部 `endpos`：

- 只求出现次数：子树求和，也就是模板中的降序累加；
- 求某个子串第一次、最后一次出现：子树维护端点最小值、最大值；
- 限制右端点区间内的出现次数：对后缀链接树做 DFS 序，再离线使用树状数组，或为每个状态合并端

- 点集合;
- 大量“子串在区间内是否出现”查询：常把端点放入可持久化线段树或合并线段树。

模式串走到状态 v 后，只有当模式长度位于该状态的合法长度区间内时，才能直接使用该状态的 `endpos` 信息；从根逐字符走完整模式时这个条件自然满足。

AC 自动机

固定小写字母表。`insert` 返回模式串的终止状态；相同模式串会得到同一个终止状态。`count_occurrences` 返回每个状态代表的模式在文本中的出现次数，允许重叠。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class AhoCorasick {
    struct Node {
        array<int, 26> next{};
        int fail = 0;
    };

    vector<Node> nodes{Node{}};
    vector<int> bfs_order;
    bool built = false;

public:
    int insert(const string& pattern) {
        assert(!built);
        int node = 0;
        for (char character : pattern) {
            int letter = character - 'a';
            assert(0 <= letter && letter < 26);
            if (nodes[node].next[letter] == 0) {
                nodes[node].next[letter] = (int)nodes.size();
                nodes.push_back(Node{});
            }
            node = nodes[node].next[letter];
        }
        return node;
    }

    void build() {
        assert(!built);
        built = true;
        queue<int> que;
        for (int letter = 0; letter < 26; ++letter) {
            int child = nodes[0].next[letter];
            if (child != 0) que.push(child);
        }
        while (!que.empty()) {
            int node = que.front();
            que.pop();
            bfs_order.push_back(node);
        }
    }
};
```

```

        for (int letter = 0; letter < 26; ++letter) {
            int child = nodes[node].next[letter];
            if (child != 0) {
                nodes[child].fail =
                    nodes[nodes[node].fail].next[letter];
                que.push(child);
            } else {
                nodes[node].next[letter] =
                    nodes[nodes[node].fail].next[letter];
            }
        }
    }
}

vector<long long> count_occurrences(const string& text) const {
    assert(built);
    vector<long long> count(nodes.size());
    int node = 0;
    for (char character : text) {
        int letter = character - 'a';
        assert(0 <= letter && letter < 26);
        node = nodes[node].next[letter];
        ++count[node];
    }
    for (auto iterator = bfs_order.rbegin();
         iterator != bfs_order.rend(); ++iterator) {
        int state = *iterator;
        count[nodes[state].fail] += count[state];
    }
    return count;
}
};

```

构建和查询总复杂度为 $O(\sum |pattern| + |text| + 26 \times states)$ 。空模式串的终止状态是根，其出现次数定义需按题意单独处理。

回文自动机（PAM）

状态 0 表示长度为 -1 的奇根，状态 1 表示长度为 0 的偶根。每加入一个字符，最多新建一个本质不同回文串状态。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class PalindromicTree {
public:
    struct Node {
        array<int, 26> next{};
        int fail = 0;
        int length = 0;
        long long occurrences = 0;
    };

```



```

};

private:
    vector<Node> nodes;
    string text;
    int last = 1;

    int suffix_link_candidate(int node, int position) const {
        while (position - nodes[node].length - 1 < 0 ||
               text[position - nodes[node].length - 1] != text[position]) {
            node = nodes[node].fail;
        }
        return node;
    }

public:
    PalindromicTree() : nodes(2) {
        nodes[0].length = -1;
        nodes[0].fail = 0;
        nodes[1].length = 0;
        nodes[1].fail = 0;
    }

    int add(char character) {
        int letter = character - 'a';
        assert(0 <= letter && letter < 26);
        text.push_back(character);
        int position = (int)text.size() - 1;
        int parent = suffix_link_candidate(last, position);
        if (nodes[parent].next[letter] == 0) {
            int created = (int)nodes.size();
            nodes.push_back(Node{});
            nodes[created].length = nodes[parent].length + 2;
            if (nodes[created].length == 1) {
                nodes[created].fail = 1;
            } else {
                int candidate =
                    suffix_link_candidate(nodes[parent].fail, position);
                nodes[created].fail = nodes[candidate].next[letter];
            }
            nodes[parent].next[letter] = created;
        }
        last = nodes[parent].next[letter];
        ++nodes[last].occurrences;
        return last;
    }

    vector<long long> occurrence_counts() const {
        vector<long long> count(nodes.size());
        for (int node = 0; node < (int)nodes.size(); ++node) {
            count[node] = nodes[node].occurrences;
        }
        for (int node = (int)nodes.size() - 1; node >= 2; --node) {

```

```

        count[nodes[node].fail] += count[node];
    }
    return count;
}

int distinct_palindrome_count() const {
    return (int)nodes.size() - 2;
}

const vector<Node>& data() const {
    return nodes;
}
};

```

构造复杂度和空间复杂度均为 $O(n|\Sigma|)$ 的定长转移存储形式（小写字母下常数为 26）。统计总出现次数时必须按回文长度从长到短向 fail 汇总；状态创建顺序恰好满足这一点。

字符串最小表示法

返回循环同构中字典序最小的旋转起点；若有多个相同答案，返回最小下标。复杂度为 $O(n)$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

int minimum_rotation(const string& text) {
    int size = (int)text.size();
    if (size == 0) return 0;
    int lhs = 0;
    int rhs = 1;
    int matched = 0;
    while (lhs < size && rhs < size && matched < size) {
        char lhs_char = text[(lhs + matched) % size];
        char rhs_char = text[(rhs + matched) % size];
        if (lhs_char == rhs_char) {
            ++matched;
            continue;
        }
        if (lhs_char > rhs_char) {
            lhs += matched + 1;
            if (lhs == rhs) ++lhs;
        } else {
            rhs += matched + 1;
            if (lhs == rhs) ++rhs;
        }
        matched = 0;
    }
    return min(lhs, rhs);
}

```

Manacher

`odd_radius[i]` 表示以 i 为中心的最长奇回文半径（包含中心），`even_radius[i]` 表示中心位于 $i-1, i$ 之间的最长偶回文半径。

```
#include <bits/stdc++.h>
using namespace std;

pair<vector<int>, vector<int>> manacher(const string& text) {
    int size = (int)text.size();
    vector<int> odd_radius(size);
    for (int center = 0, left = 0, right = -1; center < size; ++center) {
        int radius = center > right
            ? 1 : min(odd_radius[left + right - center], right - center + 1);
        while (center - radius >= 0 && center + radius < size &&
            text[center - radius] == text[center + radius]) {
            ++radius;
        }
        odd_radius[center] = radius;
        if (center + radius - 1 > right) {
            left = center - radius + 1;
            right = center + radius - 1;
        }
    }

    vector<int> even_radius(size);
    for (int center = 0, left = 0, right = -1; center < size; ++center) {
        int radius = center > right
            ? 0 : min(even_radius[left + right - center + 1],
                right - center + 1);
        while (center - radius - 1 >= 0 && center + radius < size &&
            text[center - radius - 1] == text[center + radius]) {
            ++radius;
        }
        even_radius[center] = radius;
        if (center + radius - 1 > right) {
            left = center - radius;
            right = center + radius - 1;
        }
    }
    return {odd_radius, even_radius};
}
```

最长回文长度是 $\max(2*\text{odd_radius}[i]-1, 2*\text{even_radius}[i])$ 。复杂度为 $O(n)$ 。

Z 函数

`z[i]` 是字符串与后缀 `text[i..]` 的最长公共前缀长度，约定 `z[0]=n`。

```
#include <bits/stdc++.h>
using namespace std;

vector<int> z_function(const string& text) {
```

```

int size = (int)text.size();
vector<int> z(size);
if (size == 0) return z;
z[0] = size;
for (int index = 1, left = 0, right = 0; index < size; ++index) {
    if (index <= right) {
        z[index] = min(right - index + 1, z[index - left]);
    }
    while (index + z[index] < size &&
           text[z[index]] == text[index + z[index]]) {
        ++z[index];
    }
    if (index + z[index] - 1 > right) {
        left = index;
        right = index + z[index] - 1;
    }
}
return z;
}

```

匹配模式串时可计算 `pattern + separator + text` 的 Z 函数：分隔符必须不出现在两串中。复杂度为 $O(n)$ 。

常见推论

设字符串长度为 n ：

- 长度 b 是 border，当且仅当 $z[n - b] \geq b$ ；由于该后缀只有 b 个字符，这里实际上必有等号。
- p 是周期，当且仅当 $z[p] \geq n - p$ 。从小到大找第一个满足者即可得到最小位移周期。
- 长度为 $length$ 的前缀在位置 i 出现，当且仅当 $z[i] \geq length$ ；把所有 Z 值排序、做桶计数或离线查询，就能同时回答很多前缀出现次数。
- $i + z[i] == n$ 的位置恰好对应一个 border，其长度为 $z[i]$ 。

KMP 更方便沿 border 链跳转并建立失配树； Z 函数更方便判断某个指定位置与整串前缀能匹配多长。周期问题中两者等价，选择能让后续统计更直接的一种即可。

KMP

`prefix[i]` 是 `text[0..i]` 的最长真前后缀长度。

```

#include <bits/stdc++.h>
using namespace std;

vector<int> prefix_function(const string& text) {
    vector<int> prefix(text.size());
    for (int index = 1; index < (int)text.size(); ++index) {
        int matched = prefix[index - 1];
        while (matched > 0 && text[index] != text[matched]) {
            matched = prefix[matched - 1];
        }
        if (text[index] == text[matched]) ++matched;
        prefix[index] = matched;
    }
}

```

```

    return prefix;
}

vector<int> kmp_search(const string& text, const string& pattern) {
    vector<int> positions;
    if (pattern.empty()) {
        for (int index = 0; index <= (int)text.size(); ++index) {
            positions.push_back(index);
        }
        return positions;
    }
    vector<int> prefix = prefix_function(pattern);
    int matched = 0;
    for (int index = 0; index < (int)text.size(); ++index) {
        while (matched > 0 && text[index] != pattern[matched]) {
            matched = prefix[matched - 1];
        }
        if (text[index] == pattern[matched]) ++matched;
        if (matched == (int)pattern.size()) {
            positions.push_back(index - matched + 1);
            matched = prefix[matched - 1];
        }
    }
    return positions;
}

```

复杂度为 $O(n + m)$ ，返回的匹配位置允许重叠。

border、周期与失配树

长度为 b 的 border 是同时为字符串真前缀和后缀的串。设整串长度为 n ：

- 最长 border 长度是 $\text{prefix}[n - 1]$ ；所有 border 可从它开始反复令 $b = \text{prefix}[b - 1]$ 得到。
- $p \in [1, n]$ 是一个周期（位移）当且仅当长度 $n - p$ 是 border；因此所有周期是 $n - b$ ，再加平凡周期 n 。
- 最小位移周期是 $n - \text{prefix}[n - 1]$ 。若题目要求整串由某个块完整重复，则还必须满足 $n \% \text{period} == 0$ ；不满足时最短重复块长度是 n 。
- 若长度为 n 的串同时有周期 p, q ，且 $n \geq p + q - \gcd(p, q)$ ，Fine-Wilf 定理保证 $\gcd(p, q)$ 也是周期。

失配树把前缀长度 $0..n$ 当作节点，长度 $\text{length} > 0$ 的父亲是 $\text{prefix}[\text{length} - 1]$ ：

- 一个前缀的所有 border 正是它到根路径上的祖先；
- 节点 u 的子树对应所有“以长度 u 前缀为后缀”的字符串前缀，因此子树大小就是该前缀在整串中的出现次数；
- 两个前缀节点在失配树上的 LCA，是它们最长的公共 border（这里允许某个前缀本身）。

不用显式建树也能统计每个前缀的出现次数：先对每个位置 i 执行 $++\text{count}[\text{prefix}[i]]$ ，再让 length 从 n 降到 1，执行 $\text{count}[\text{prefix}[\text{length} - 1]] += \text{count}[\text{length}]$ ，最后给 $\text{count}[1..n]$ 各加 1，补上前缀自身那次出现。

若要对大量字符做 KMP 转移，可以预处理状态 $\text{matched} = 0..m$ 遇到每个字符后的新状态，得到失配自动机；状态间跳转仍然完全由 prefix 链决定。模式串自匹配、统计每个前缀出现次数、在失配树上做树上查询，是这棵树最常见的三类用途。

后缀数组与 LCP

返回所有后缀起点的字典序排列，点编号为 0-based。实现会追加值为 0 的哨兵，因此输入字符串不能包含空字符。

```
#include <bits/stdc++.h>
using namespace std;

pair<vector<int>, vector<int>> suffix_array_and_lcp(const string& text) {
    string extended = text;
    extended.push_back(char(0));
    int size = (int)extended.size();

    vector<int> suffix_array(size);
    vector<int> rank(size);
    {
        vector<int> count(max(256, size));
        for (unsigned char character : extended) ++count[character];
        for (int value = 1; value < (int)count.size(); ++value) {
            count[value] += count[value - 1];
        }
        for (int index = size - 1; index >= 0; --index) {
            unsigned char character = extended[index];
            suffix_array[--count[character]] = index;
        }
        int class_count = 1;
        for (int position = 1; position < size; ++position) {
            if (extended[suffix_array[position]] !=
                extended[suffix_array[position - 1]]) {
                ++class_count;
            }
            rank[suffix_array[position]] = class_count - 1;
        }
    }

    vector<int> shifted(size);
    vector<int> next_rank(size);
    for (int length = 1; length < size; length <= 1) {
        for (int position = 0; position < size; ++position) {
            shifted[position] = suffix_array[position] - length;
            if (shifted[position] < 0) shifted[position] += size;
        }
        int class_count = *max_element(rank.begin(), rank.end()) + 1;
        vector<int> count(class_count);
        for (int index : shifted) ++count[rank[index]];
        for (int value = 1; value < class_count; ++value) {
            count[value] += count[value - 1];
        }
        for (int position = size - 1; position >= 0; --position) {
            int index = shifted[position];
            suffix_array[--count[rank[index]]] = index;
        }

        next_rank[suffix_array[0]] = 0;
    }
}
```

```

class_count = 1;
for (int position = 1; position < size; ++position) {
    int current = suffix_array[position];
    int previous = suffix_array[position - 1];
    pair<int, int> current_pair{
        rank[current], rank[(current + length) % size]};
    pair<int, int> previous_pair{
        rank[previous], rank[(previous + length) % size]};
    if (current_pair != previous_pair) ++class_count;
    next_rank[current] = class_count - 1;
}
rank.swap(next_rank);
if (class_count == size) break;
}

suffix_array.erase(suffix_array.begin());
int text_size = (int)text.size();
vector<int> suffix_rank(text_size);
for (int position = 0; position < text_size; ++position) {
    suffix_rank[suffix_array[position]] = position;
}

vector<int> lcp(text_size);
int matched = 0;
for (int index = 0; index < text_size; ++index) {
    int position = suffix_rank[index];
    if (position == 0) continue;
    int previous = suffix_array[position - 1];
    while (index + matched < text_size &&
           previous + matched < text_size &&
           text[index + matched] == text[previous + matched]) {
        ++matched;
    }
    lcp[position] = matched;
    if (matched > 0) --matched;
}
return {suffix_array, lcp};
}

```

$lcp[i]$ 是 $suffix_array[i-1]$ 与 $suffix_array[i]$ 所指后缀的最长公共前缀, $lcp[0]=0$ 。构造复杂度为 $O(n \log n)$, 空间复杂度为 $O(n)$; 任意两个后缀的 LCP 可转化为 lcp 区间最小值查询。

Lyndon 分解

Duval 算法把字符串唯一分解成字典序非增的 Lyndon 串序列, 复杂度为 $O(n)$ 。

```

#include <bits/stdc++.h>
using namespace std;

vector<string> lyndon_factorization(const string& text) {
    vector<string> factors;
    int size = (int)text.size();

```

```

for (int begin = 0; begin < size; ) {
    int scan = begin + 1;
    int matched = begin;
    while (scan < size && text[matched] <= text[scan]) {
        matched = text[matched] < text[scan] ? begin : matched + 1;
        ++scan;
    }
    int factor_length = scan - matched;
    while (begin <= matched) {
        factors.push_back(text.substr(begin, factor_length));
        begin += factor_length;
    }
}
return factors;
}

```

通配符

定义字符串下标从 0 开始，给定长度为 n 的文本串 A 和长度为 m 的模式串 B 。先把字符映射为整数，考虑单个位置的差 $A(x) - B(y)$ 。

若直接把各位置的差相加，不同位置可能正负抵消，例如 ab 与 ba 会被错误地判为相等。改为累加平方差：

$$F(x) = \sum_{i=0}^{m-1} (A(x - m + 1 + i) - B(i))^2.$$

在整数精确运算且没有溢出的前提下， $F(x) = 0$ 当且仅当对应子串完全匹配。

我们考虑翻转一下 B 串，令其为 S ，则有 $B(i) = S(m - 1 - i)$ 。

$$F(x) = \sum_{i=0}^{m-1} (A(x - m + 1 + i) - S(m - 1 - i))^2$$

$$F(x) = \sum_{i=0}^{m-1} S(m - 1 - i)^2 + \sum_{i=0}^{m-1} A(x - m + 1 + i)^2 - 2 \times \sum_{i=0}^{m-1} A(x - m + 1 + i)S(m - 1 - i)$$

第一项是一个定值，第二项可以 $O(n)$ 预处理，然后前缀和 $O(1)$ 得到，第三项不难发现是一个卷积的形式，所以可以通过 FFT 得到，整体复杂度 $O(n \log n)$ 。

普通精确匹配应优先使用线性的 KMP 或 Z Algorithm。卷积方法的价值在于处理通配符。令通配符编码为 0，普通字符编码为非零整数，并定义

$a*b$
 $aeb*ob$

$$f(x, y) = (A(x) - B(y))^2 A(x) B(y).$$

只要任意一侧是通配符，该项就是 0；两侧都是普通字符时，该项非负，且仅在字符相同时为 0。同样翻转 B 得到 S ：

$$\begin{aligned}
F(x) &= \sum_{i=0}^{m-1} (A(x-m+1+i) - S(m-1-i))^2 A(x-m+1+i) S(m-1-i) \\
&= \sum_{i=0}^{m-1} A(x-m+1+i) S(m-1-i)^3 + \sum_{i=0}^{m-1} A(x-m+1+i)^3 S(m-1-i) - 2 \times \sum_{i=0}^{m-1} A(x-m+1+i)^2 S(m-1-i)
\end{aligned}$$

三个求和项都是卷积，可用三次卷积得到全部对齐位置，复杂度 $O((n+m) \log(n+m))$ 。

使用浮点 FFT 时必须估计系数上界并处理舍入；使用 NTT 时，若真实整数和可能达到或超过模数，单模数下的“结果为零”可能是假阳性，应使用足够大的模数乘积、多个模数或随机编码降低风险。本仓库的 NTT 见 `math/16-polynomial.md`。

广义后缀自动机

广义后缀自动机维护多个字符串的全部子串。这里先把所有字符串插入 Trie，再按 BFS 顺序把 Trie 边加入 SAM；这样不会产生跨越两个字符串边界的子串，也能正确处理重复字符串和公共前缀。

固定小写字母表，状态 0 是初始状态。必须先完成所有 `insert`，再调用一次 `build`。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class GeneralizedSuffixAutomaton {
public:
    struct State {
        array<int, 26> next{};
        int link = -1;
        int length = 0;
    };

private:
    struct TrieNode {
        array<int, 26> next{};
    };

    vector<TrieNode> trie{TrieNode{}};
    vector<State> states;
    bool built = false;

    int extend_from(int last, int letter) {
        int existing = states[last].next[letter];
        if (existing != 0) {
            if (states[last].length + 1 == states[existing].length) {
                return existing;
            }
            int clone = (int)states.size();
            states.push_back(states[existing]);
            states[clone].length = states[last].length + 1;
            int prefix = last;
            while (prefix != -1 &&

```

```

        states[prefix].next[letter] == existing) {
            states[prefix].next[letter] = clone;
            prefix = states[prefix].link;
        }
        states[existing].link = clone;
        return clone;
    }

    int current = (int)states.size();
    states.push_back(State{});
    states[current].length = states[last].length + 1;
    int prefix = last;
    while (prefix != -1 && states[prefix].next[letter] == 0) {
        states[prefix].next[letter] = current;
        prefix = states[prefix].link;
    }
    if (prefix == -1) {
        states[current].link = 0;
        return current;
    }

    int target = states[prefix].next[letter];
    if (states[prefix].length + 1 == states[target].length) {
        states[current].link = target;
        return current;
    }

    int clone = (int)states.size();
    states.push_back(states[target]);
    states[clone].length = states[prefix].length + 1;
    while (prefix != -1 && states[prefix].next[letter] == target) {
        states[prefix].next[letter] = clone;
        prefix = states[prefix].link;
    }
    states[target].link = clone;
    states[current].link = clone;
    return current;
}

public:
    void insert(const string& text) {
        assert(!built);
        int node = 0;
        for (char character : text) {
            int letter = character - 'a';
            assert(0 <= letter && letter < 26);
            if (trie[node].next[letter] == 0) {
                trie[node].next[letter] = (int)trie.size();
                trie.push_back(TrieNode{});
            }
            node = trie[node].next[letter];
        }
    }
}

```

```

void build() {
    assert(!built);
    built = true;
    states = {State{}};
    vector<int> trie_state(trie.size());
    queue<int> que;
    que.push(0);
    while (!que.empty()) {
        int node = que.front();
        que.pop();
        for (int letter = 0; letter < 26; ++letter) {
            int child = trie[node].next[letter];
            if (child == 0) continue;
            trie_state[child] = extend_from(trie_state[node], letter);
            que.push(child);
        }
    }
}

bool contains(const string& pattern) const {
    assert(built);
    int state = 0;
    for (char character : pattern) {
        int letter = character - 'a';
        assert(0 <= letter && letter < 26);
        state = states[state].next[letter];
        if (state == 0) return false;
    }
    return true;
}

long long distinct_substring_count() const {
    assert(built);
    long long answer = 0;
    for (int state = 1; state < (int)states.size(); ++state) {
        answer += states[state].length -
            states[states[state].link].length;
    }
    return answer;
}

vector<long long> occurrence_counts(
    const vector<string>& source_strings) const {
    assert(built);
    vector<long long> count(states.size());
    for (const string& text : source_strings) {
        int state = 0;
        for (char character : text) {
            int letter = character - 'a';
            assert(0 <= letter && letter < 26);
            state = states[state].next[letter];
            assert(state != 0);
            ++count[state];
        }
    }
}

```

```

    }
}

vector<int> order(states.size());
iota(order.begin(), order.end(), 0);
sort(order.begin(), order.end(), [](int lhs, int rhs) {
    return states[lhs].length > states[rhs].length;
});
for (int state : order) {
    if (states[state].link != -1) {
        count[states[state].link] += count[state];
    }
}
return count;
}

long long occurrences_of(const string& pattern,
                        const vector<long long>& count) const {
    assert(built && count.size() == states.size());
    int state = 0;
    for (char character : pattern) {
        int letter = character - 'a';
        assert(0 <= letter && letter < 26);
        state = states[state].next[letter];
        if (state == 0) return 0;
    }
    return count[state];
}

const vector<State>& data() const {
    assert(built);
    return states;
}
};

```

Trie 与 SAM 的状态数都是所有字符串总长度的 $O(\sum |s_i|)$ 量级，构造为线性复杂度（这里为固定 26 字母转移）。`distinct_substring_count` 统计所有字符串的非空子串并集。

出现次数必须在自动机构造完成后，把原字符串逐个从根重新扫描，再沿后缀链接按 `length` 降序累加；不能把 Trie 结点数或 SAM 克隆次数直接当成出现次数。`source_strings` 应与建 Trie 时的字符串集合一致，重复字符串会重复贡献出现次数。当前实现为简洁使用比较排序，因此出现次数统计为 $O(L \log L + \sum |s_i|)$ 。

双模字符串哈希

只保留最常用的功能： $O(n)$ 预处理， $O(1)$ 取得半开区间 `[left, right)` 的哈希。比较两个子串时直接比较 `get` 的返回值。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct StringHash {
    static constexpr int base = 911382323;

```

```

static constexpr array<int, 2> mod{1000000007, 1000000009};

array<vector<int>, 2> hash;
array<vector<int>, 2> power;

explicit StringHash(string_view text) {
    for (int k = 0; k < 2; ++k) {
        hash[k].resize(text.size() + 1);
        power[k].resize(text.size() + 1, 1);
        for (int i = 0; i < (int)text.size(); ++i) {
            int value = (unsigned char)text[i] + 1;
            hash[k][i + 1] = (int)(
                ((long long)hash[k][i] * base + value) % mod[k]);
            power[k][i + 1] = (int)(
                (long long)power[k][i] * base % mod[k]);
        }
    }
}

array<int, 2> get(int left, int right) const {
    assert(0 <= left && left <= right && right < (int)hash[0].size());
    array<int, 2> result{};
    for (int k = 0; k < 2; ++k) {
        long long value = hash[k][right] -
            (long long)hash[k][left] * power[k][right - left] % mod[k];
        if (value < 0) value += mod[k];
        result[k] = (int)value;
    }
    return result;
}
};

```

同一组字符串必须使用相同的底数和模数。双模碰撞概率很低，但不是严格零；要求确定性时应使用后缀数组、Z/KMP 或自动机。字符按 `unsigned char` 编码，所以 UTF-8 中文按字节处理。

LCP 不需要写进结构体：对长度二分并比较两次 `get` 即可，复杂度 $O(\log n)$ 。拼接哈希的公式是 `left_hash * power[right_length] + right_hash`；只有题目确实需要拼接时再写两行，公共模板不为它增加接口。

本模板保留字符顺序。若题目需要集合切换、区间出现次数奇偶或图割中让内部边抵消，见 `13-xor-hash.md`；异或哈希不能替代这里的字符串哈希。

异或哈希与可交换哈希

多项式字符串哈希保留顺序，而异或哈希故意忽略顺序。给每个对象 x 分配一个随机 64 位标签 $h(x)$ ，集合签名定义为

$$H(S) = \bigoplus_{x \in S} h(x).$$

插入、删除或切换一个对象都只需再异或一次它的标签。这就是 Zobrist hashing 的基本形式。

```

#include <bits/stdc++.h>
using namespace std;

```

```

using u64 = unsigned long long;

u64 splitmix64(u64 value) {
    value += 0x9e3779b97f4a7c15ULL;
    value = (value ^ (value >> 30)) * 0xbf58476d1ce4e5b9ULL;
    value = (value ^ (value >> 27)) * 0x94d049bb133111ebULL;
    return value ^ (value >> 31);
}

u64 random_tag(u64 value) {
    static const u64 seed = (u64)
        chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(value + seed);
}

```

同一次程序中必须让相同对象调用得到相同标签，因此 `seed` 只能初始化一次。运行时随机种子可以避免输入方提前针对固定哈希构造数据；它仍是概率算法，不用于密码学。

最重要的边界：XOR 只保留奇偶

因为

$$h(x) \oplus h(x) = 0,$$

直接异或一串元素得到的不是多重集哈希，而是“出现奇数次的元素集合”的哈希：

- 判断两个集合是否相同：可以异或，但必须保证每个元素在集合中最多保留一次；
- 动态 `toggle(x)`：非常适合，执行 `signature ^= random_tag(x)`；
- 判断区间中是否每个值都出现偶数次：适合使用前缀异或哈希；
- 判断两个多重集或两个字符串是否互为排列：不能只用 XOR，偶数次重复会被消掉。

多重集需要使用可逆的可交换运算，例如对随机标签按模 2^{64} 求和：

```

u64 multiset_hash = 0;
multiset_hash += random_tag(value); // 插入一次
multiset_hash -= random_tag(value); // 删除一次

```

无符号整数溢出就是模 2^{64} ，定义良好。需要更低碰撞概率时使用两个独立种子得到一对 64 位签名。

前缀 XOR：区间奇偶集合

```

vector<u64> prefix(values.size() + 1);
for (int i = 0; i < (int)values.size(); ++i) {
    prefix[i + 1] = prefix[i] ^ random_tag(values[i]);
}

// [left, right) 中所有出现奇数次的值的随机签名。
u64 range_hash = prefix[right] ^ prefix[left];

```

典型结论：

- `range_hash == 0` 表示区间内每个值都出现偶数次，结论是概率正确；
- 两个区间哈希相同，表示它们“奇数次元素集合”相同，不表示完整频率相同；
- 两个集合签名异或后，得到它们对称差的签名。

绝不能直接异或原值。原值之间可能满足人为构造的线性关系，例如 $a \oplus b \oplus c = 0$ ；先经过带随机种子的 `random_tag` 才能把这种结构打散。

图的割边集合签名

给每条无向边分配随机标签，并把它异或进两个端点：

```
vector<u64> vertex_hash(node_count);
for (int edge_id = 0; edge_id < (int)edges.size(); ++edge_id) {
    auto [u, v] = edges[edge_id];
    u64 tag = random_tag(edge_id);
    vertex_hash[u] ^= tag;
    vertex_hash[v] ^= tag;
}
```

对任意点集 S 异或其中所有 `vertex_hash`：内部边出现两次而抵消，只剩下割 $\delta(S)$ 中跨出点集的边标签异或和。常见用途是树上子树割、随机检测某个割是否为空，以及“割中恰有一条边”时借助 `tag -> edge_id` 映射找出候选边；取出候选后仍应检查它是否真的跨越该割。

无序树和多重子结构不能裸 XOR

无序根树的儿子是一个多重集。若直接写

```
subtree_hash ^= random_tag(child_hash);
```

两个相同儿子会完全抵消，得到明显错误的结构信息。应使用求和等保留重数的可交换组合，例如

```
subtree_hash += random_tag(child_hash);
```

再把儿子数量、当前结点颜色或一个额外随机盐混入结果。不同类型的对象也应先编码成不同 `key`：例如点 7、颜色 7、边 7 不应无意中使用同一个标签。

什么时候不要用

- 普通字符串或序列相等需要保留顺序，使用多项式哈希、后缀数组、KMP/Z 或自动机；
- 题目要求绝对确定性时不能用随机哈希；
- 对手能看到固定种子并自适应构造数据时，单个 64 位签名不够可靠；
- 哈希相等只能当作候选判定，能以较低代价复核时应始终复核。

异或哈希的价值不是“比普通哈希更强”，而是它支持 $O(1)$ 撤销，并能让出现两次的对象自动抵消；只有题目恰好需要这两个代数性质时才使用。